UM Hackathon 2026

Domain 1: AI Systems & Agentic Workflow Automation

Resolve Mesh: System Analysis Documentation

Prepared by Unemployed Squad:

- Sean Lee Jing Xuan
- Quah Jia Hang
- Tan Jin Xiang
- Teng Wen Hong
- Tan Hui Zhe

1. Introduction:

ResolveMesh introduces a strategic, AI-driven solution to the high-friction problem of manual dispute management and data privacy risks. It addresses the core inefficiencies in customer claims processing by replacing linear, labor-intensive workflows with an autonomous, multi-agent orchestrator.

1.1. Purpose:

This System Analysis Document highlights the technical scope and architectural decisions governing the development of ResolveMesh. The key system elements covered in this documentation include:

1. Architecture:
    a. A hybrid-cloud infrastructure utilizing a FastAPI backend and n8n Cloud orchestration.
    b. The system operates through decoupled AI agents that function as independent micro-services to ensure scalability and modularity.
2. Data Flows:
    a. Analysis of how dispute data moves from the Customer Ingestion Portal through a Privacy Shield (Spacy), is processed by Z.ai and GPT-5-mini agents, and is stored persistently in Supabase
3. Model Process:
    a. Visualization of the end-to-end "Golden Path": from initial claim ingestion and PII redaction to forensic triangulation and the final resolution verdict.
4. Role of Reference:
    a. This document serves as the technical blueprint for the Development Team and QA Team to ensure system integrity and cross-platform compatibility.

1.2. Background:

Current industry standards for dispute resolution rely on manual oversight, which is slow, prone to human error, and often neglects strict data privacy compliance.

1. Previous Version (Legacy Systems):
    - Traditional solutions are often operated as siloed ticket systems or manual email-based workflows.
    - These legacy methods require significant labor and lack the capability to autonomously verify evidence across multiple digital ledgers.
2. Changes in Major Architectural Components
    - Transition from manual processing to an Agentic AI Orchestration model using n8n and Z.ai.

3. New capabilities that are only introduced in this
    - Introduction of autonomous PII Redaction, multi-modal Evidence Analysis (GPT-5-mini), and real-time Forensic Audit Logging.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Customer	Submits disputes via the portal and provides evidence (receipts/logs).	A fast, fair resolution process with guaranteed data privacy (PII masking).
Customer Service Department Staff	Reviews AI-generated verdicts and manages the "Human-in-the-Loop" approval.	Clear forensic evidence, reduced manual workload, and scannable "Police-Friendly" reports.
Dev Team	Build and maintain the agentic workflows, API, and dashboard.	Modular architecture, well-documented API contracts, and scalable infrastructure.
QA Team	Validates system behavior across various dispute edge cases.	Detailed system logs, defined test boundaries, and a measurable AI accuracy threshold.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

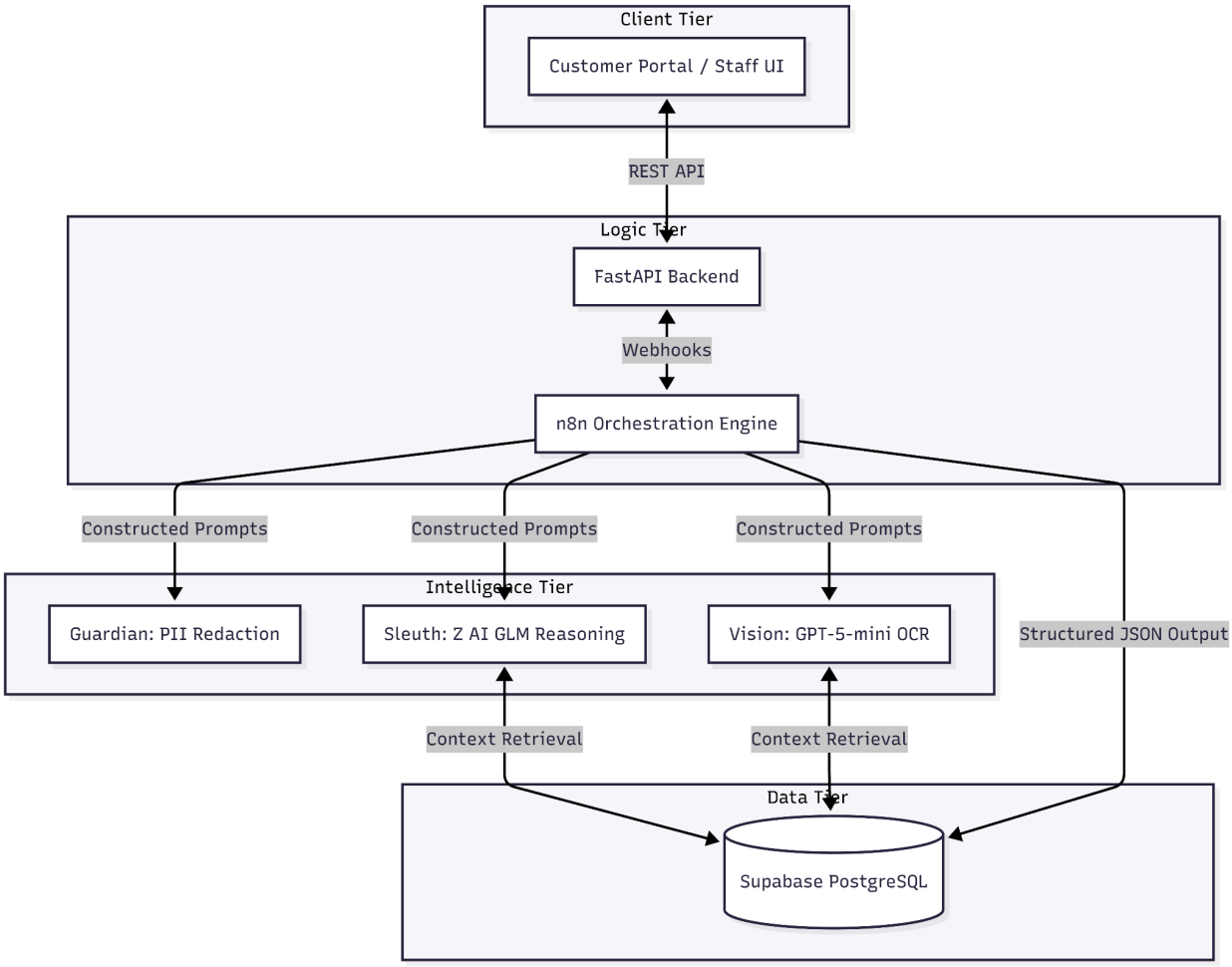
Type	Details
System	Web-based Dashboard (React)
Architecture	Hybrid-Cloud with Multi-Agent Orchestration

ResolveMesh is structured as a client-server system featuring two primary interfaces: the Customer Ingestion Portal (developed for initial data capture) and the Staff Audit Dashboard. These clients communicate via a FastAPI backend layer, which serves as the entry point for the n8n orchestration engine. The system utilizes Supabase for persistent storage of transaction logs and dispute states, ensuring high data integrity and real-time updates.

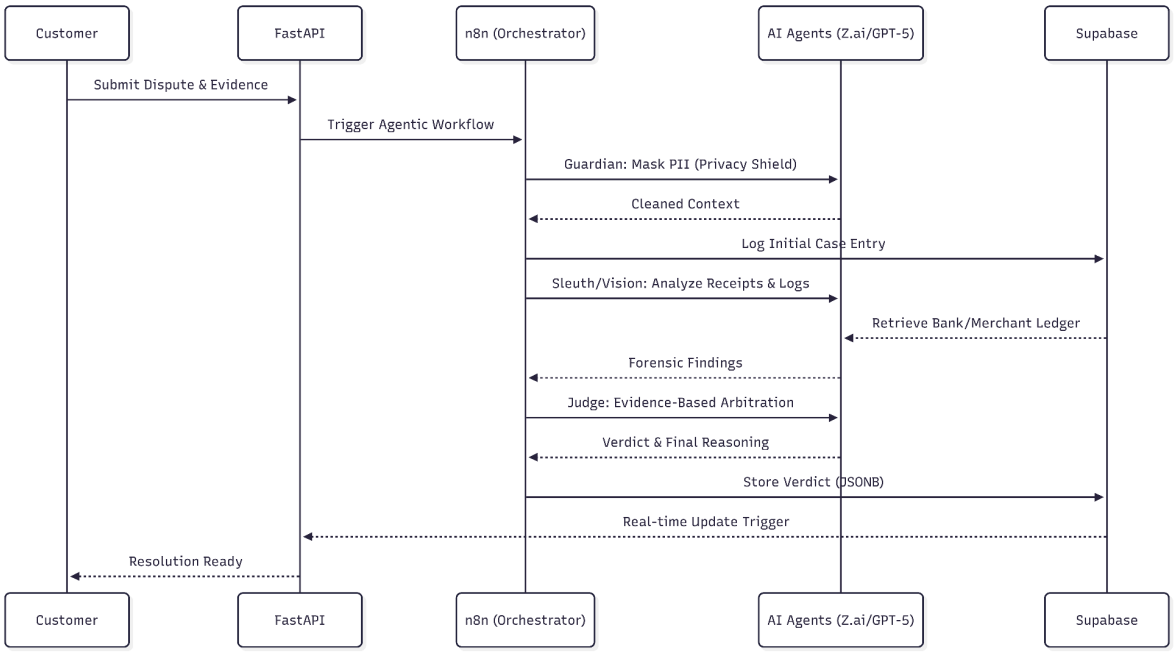
2.1.2 LLM as Service Layer

The Z AI GLM and GPT-5-mini models are integrated not as static modules, but as an active Service Layer orchestrated by n8n. The system constructs dynamic prompts using multi-step agentic logic, where the output of the "Guardian" (PII mask) becomes the context for the "Sleuth" (investigation). Input chunking and token limitations are enforced at the FastAPI level before data reaches the n8n workflow to prevent context window overflow.

2.1.3 Dependency Diagram



2.1.4 Sequence Diagram



2.2. Technological Stack

2.2.1. **Frontend:** React + Vite + Tailwind CSS (Optimized for real-time audit dashboards).

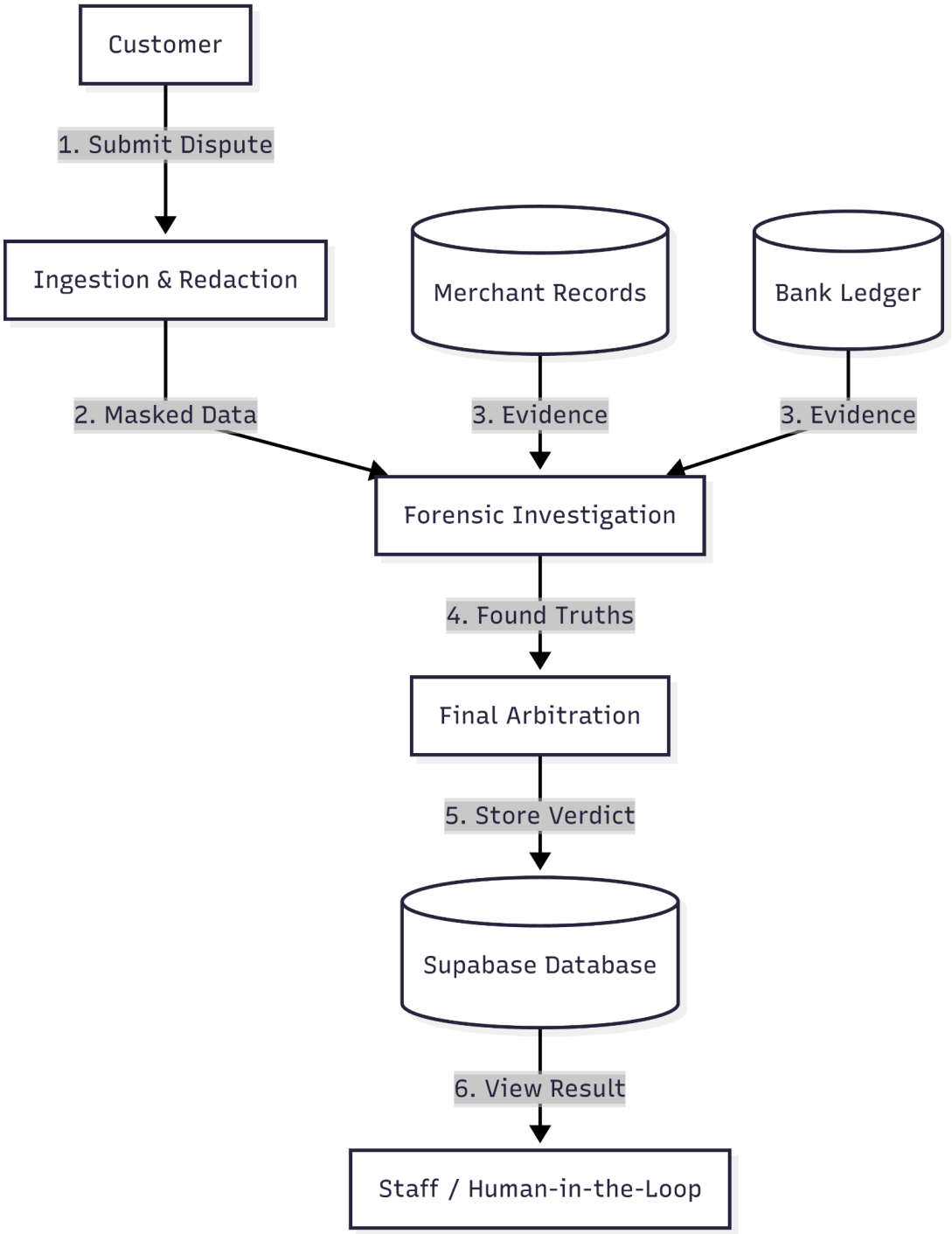
2.2.2. **Backend:** FastAPI (Python) - Serves as the high-performance RESTful bridge for microservices.

2.2.3. **Database:** Supabase (PostgreSQL) - Relational storage for disputes, utilizing JSONB for flexible agent reports.

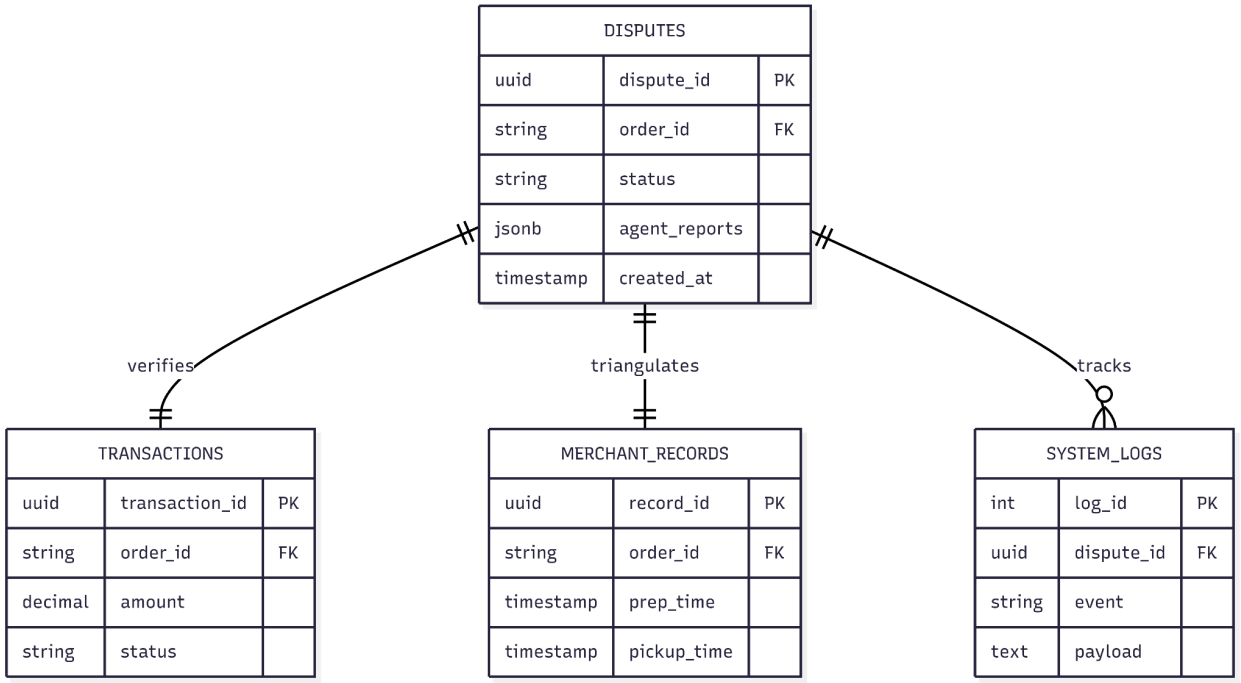
2.2.4. **Orchestration**: n8n Cloud - Manages multi-agent logic and API handshakes.

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. The following core features represent the primary functionality to be demonstrated by the end of UMHackathon 2026:

#	Features	Description
1	Privacy-First Ingestion	Customers submit disputes via text or image; the "Guardian" agent autonomously masks PII (names, IDs) to ensure PDPA compliance before processing.
2	Multi-Agent Investigation	The "Sleuth" and "Vision" agents autonomously retrieve and cross-reference data from merchant logs and internal transaction ledgers to identify discrepancies.

3	Automated Arbitration	The "Judge" agent weighs findings against predefined company policy to generate a logical verdict and reasoning.
4	Forensic Reporting	The "Reporter" agent generates a downloadable PDF audit trail and a redacted police-friendly supplement for the customer.

3.2. Non-Functional Requirements (NFRs)

These qualities ensure the system remains reliable, secure, and efficient under operational conditions:

Quality	**Requirements**	**Implementation**
Privacy & Security	Ensure zero leakage of raw PII to the LLM's permanent memory or unauthorized logs.	Implement a dedicated Spacy NLP redaction layer in the backend before data reaches the n8n orchestrator.
Reliability	100% data persistence for dispute states to prevent "orphaned" cases during agentic loops.	Use Supabase real-time listeners and atomic database transactions to track state transitions.
Token Latency	AI-generated responses for investigation should return within 5s for better UX.	Implement asynchronous API calls in FastAPI with a user-facing loading state during the n8n handshake.

Cost Efficiency	Average token cost per session must remain below a defined budget threshold.	Use fixed-length system prompts, truncate oversized inputs, and cache repeating ledger queries for 24 hours.
Maintainability	The system must allow for independent updates to specific agent logic without breaking the pipeline.	Decouple agent logic into individual n8n workflow nodes and maintain independent backend microservices.

3.3. Out of Scope / Future Enhancements

High-level features planned for the "Grand Vision" of ResolveMesh that are not part of the current hackathon MVP:

- Live Dispute Mediation: An in-app, AI-moderated chat room between the customer and merchant staff for real-time clarification.
- Predictive Fraud Analytics: Using historical dispute data to proactively flag high-risk accounts before a dispute is even filed.
- Multi-Language Support: Expanding the "Guardian" agent's NLP capabilities to support Bahasa Melayu, Mandarin, and Tamil for local inclusivity.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

The team's deployment strategy utilizes a grayscale rollout to mitigate risk. New features, such as updated "Judge" agent reasoning models, are initially released to 5% of administrative users to monitor verdict consistency before a gradual 100% rollout. Following a successful rollout, A/B testing is conducted to compare different system prompts for the "Sleuth" agent to determine which variant optimizes investigation speed and accuracy.

4.1.2. Emergency Rollback (ER) & Golden Release:

In the event of critical anomalies, such as detected PII leakage or system-wide agent timeouts, an Emergency Rollback (ER) is automatically triggered. The automated pipeline is configured to immediately redeploy the "Golden Release"—the most recent stable version of the application known to meet all security and performance benchmarks.

4.1.3. Priority Matrix:

Priority Level	**Trigger Condition**	**Required Action**
P1 - Critical	Detected PII leakage in agent outputs or a failure rate > 1% in automated decisions.	Immediate Emergency Rollback to the Golden Release and notify the Security Lead.
P2 - High	System-wide latency exceeding 5 seconds for agentic reasoning loops.	Initiate load balancing and scale microservice instances to handle traffic.
P3 - Medium	Visual formatting errors in the "Police-Friendly" PDF reports.	Log the defect and assign it for a hotfix in the next sprint cycle.

4.2. Monitoring:

The monitoring plan involves continuous tracking of system health through n8n execution logs and Supabase real-time event monitoring. If the system detects a breach of defined thresholds — such as a 5xx error from an AI API or database connection instability — an automated alert is pushed to the Development Team via integrated webhooks to trigger immediate diagnosis.

4.3. Assumptions:

Several key operational and environmental conditions are assumed to be valid for the successful deployment and function of ResolveMesh:

(a) Internet Stability: Users are assumed to have a stable internet connection to facilitate the uploading of high-resolution evidence and receipt images.
(b) Hardware Access: Merchant staff are assumed to have authorized devices capable of accessing the web-based Audit Dashboard.
(c) Cloud Availability: It is assumed that third-party cloud providers (n8n and Supabase) maintain a minimum uptime of 99.9%.

4.4. External Dependencies:

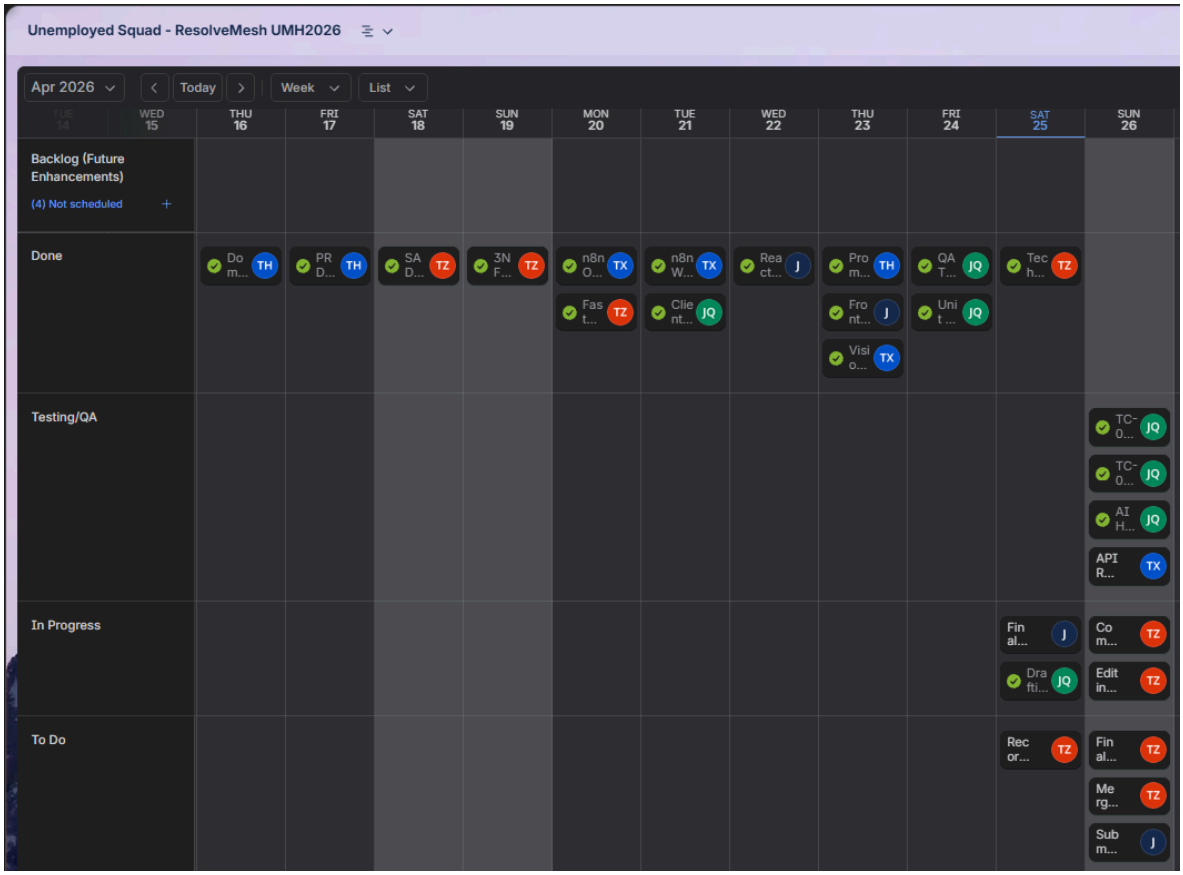
ResolveMesh relies on several external tools and APIs to fulfill its core functionalities:

Tools	Purpose	Risks
Ilmu Z.AI GLM-5.1 API	Serves as the core LLM reasoning engine for multi-step agentic investigation and arbitration.	High: API rate limits or downtime can cause core product failure. Mitigation: Implement a graceful degradation state.
GPT-5-mini	Powers multimodal vision tasks, specifically OCR for receipt analysis and file processing.	Medium: Potential for visual hallucinations in low-quality images. Mitigation: Implement high-confidence threshold checks.
n8n Cloud	The central orchestrator that manages the workflow between the API, Database, and AI agents.	High: Connectivity issues between local backend and cloud n8n. Mitigation: Maintain a local n8n instance for redundancy.
Supabase	Provides the primary PostgreSQL database and real-time data persistence layer.	Medium: Potential database latency during peak dispute periods. Mitigation: Optimized indexing for transaction lookups.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:



5.2. Team Members Role:

Team Member	Official Role	Key Responsibilities +1
Tan Jin Xiang	Technical Lead	Designed and engineered the entire n8n orchestration workflow, managing the multi-agent logic and API handshakes.
Tan Hui Zhe	Project Manager & System Analyst	Developed the FastAPI backend, managed the 10-day sprint cycle, and drafted the core system architecture/SAD.
Teng Wen Hong	Product & AI Strategist	Defined the Z AI GLM reasoning logic, and established agentic prompting strategies.
Sean Lee Jing Xuan	Frontend Lead	Developed the ResolveMesh high-fidelity React dashboard and integrated real-time system logs.
Quah Jia Hang	QA & Client-Side Developer	Led the QATD, defined client-side User Stories/Data Inputs, and developed the Risk Assessment Matrix.

5.3. Recommendations:

To enhance the future scalability, performance, and reliability of the ResolveMesh ecosystem beyond the hackathon, the following technical optimizations are suggested.

- Scalability (Distributed Caching): Implement Redis Cache for frequently accessed merchant ledger data to reduce the frequency of API calls and decrease system latency.
- Performance (Vector Database): Integrate a Vector Database (e.g., Pinecone) to allow the "Sleuth" agent to perform semantic searches over unstructured PDF evidence more efficiently.
- Reliability (CI/CD Automation): Incorporate automated regression testing gates within the GitHub Actions pipeline to ensure that new agent prompts do not degrade verdict accuracy.
- Security (Hardware Security Modules): Utilize cloud-based HSMs for managing API keys and sensitive encryption tokens used during the PII redaction phase.